Database operations in production

PostgreSQL replication and failover, backups and point in time recovery, connection pooling, zero downtime migrations, Redis and MongoDB operations, and database incident runbooks.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. PostgreSQL replication and high availability (13 min)
2. Backups, point in time recovery and restore testing (13 min)
3. Connection pooling and capacity (12 min)
4. Zero downtime migrations and upgrades (13 min)
5. Redis and MongoDB operations (12 min)
6. Database incidents and runbooks (12 min)
Cheat sheet

Lesson 1. PostgreSQL replication and high availability

13 minutes. Understand streaming replication, synchronous and asynchronous trade-offs, failover tools such as Patroni, and how to avoid split brain and data loss.

A single database server is a single point of failure. Replication keeps copies, and a failover process promotes one when the primary dies. The hard part is doing it without losing data or ending up with two primaries.

What you will learn

- Write ahead log and streaming replication
- Asynchronous versus synchronous replication
- Replication lag and what it means for reads
- Automatic failover with Patroni and a consensus store
- Split brain, fencing and promotion safety

WAL and streaming

PostgreSQL writes every change to the write ahead log first. A standby connects to the primary and replays that log continuously. The standby is a read only copy that can serve queries and take over. Replication slots keep the log until each standby has received it, but a stuck standby can fill the primary disk with retained log.

Asynchronous or synchronous

In asynchronous replication the primary commits without waiting, so it is fast but a crash can lose the last few transactions. Synchronous replication waits for at least one standby, which avoids loss but adds latency and can block writes if the standby is gone. Quorum settings such as `ANY 1 (s1, s2)` give a compromise.

Lag

Replication lag is the delay between the primary and a standby. Measure it in bytes and in seconds. Reads from a lagging standby can return old data, so route read your writes queries to the primary or wait for a log position. Alert on lag before it becomes minutes.

Automatic failover

Patroni runs next to each PostgreSQL instance and keeps a leader key in a consensus store such as etcd. If the leader key expires, the healthiest standby is promoted and the others follow it. Applications connect through a proxy or a virtual address that points to the current leader.

Split brain and fencing

Split brain means two nodes both accept writes. Prevent it with a consensus store, a watchdog that restarts a node that cannot reach it and a proxy that checks the role. After a failover, the old primary must not rejoin as primary: it needs to be rewound or rebuilt from the new one.

Diagram: PostgreSQL replication and high availability

Part	What it does	What goes wrong
Application	Writes and reads.	Hard coded host.
Primary	Accepts writes.	Crash.
Standby	Replays WAL.	Lag.
Consensus store	Holds the leader key.	Quorum lost.

Flow: Application to Primary / Primary to Standby / Standby to Consensus store

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A primary that filled its disk with retained logs

Incident: *A forgotten replica made the primary retain WAL until the disk was full.*

```
$ psql -c "select slot_name, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_ \
    lsn)) as retained from pg_replication_slots"
 slot_name | active | retained
 old_rep_2 | f      | 412 GB
(this is a simulated illustration, not a real incident)
```

A standby decommissioned weeks earlier still had a replication slot, so the primary kept every log segment since then. Dropping the slot freed the space, and the team added an alert on retained WAL per slot and a step in the decommission checklist.

Common mistakes

- Forgotten replication slots
- Synchronous replication with a single standby
- No fencing of the old primary
- Reading from standbys without considering lag
- Failover never tested

Check yourself

What does a standby replay?

Answer: The write ahead log from the primary

What is a risk of asynchronous replication?

Answer: Losing the last transactions on a crash

What prevents split brain?

Answer: A consensus store, watchdog and fencing

Why alert on replication slots?

Answer: An inactive slot retains log and can fill the disk

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Backups, point in time recovery and restore testing

13 minutes. Plan backups for a recovery point and time objective, use base backups with WAL archiving for point in time recovery and test restores on a schedule.

Replication is not a backup: it copies mistakes and deletions instantly. Backups with point in time recovery let you return to the moment before a bad change, if you have tested them.

What you will learn

- Logical versus physical backups
- Base backup plus WAL archiving
- Point in time recovery in practice
- Tools: pgBackRest and cloud snapshots
- Restore drills and monitoring

Logical and physical

A logical backup with `pg_dump` exports data as SQL and is portable across versions but slow to restore on large databases. A physical backup copies the data files and is fast to restore. Large production databases rely on physical backups, with dumps for specific tables or version moves.

Base plus WAL

A base backup is a full copy of the data directory. Continuous archiving then copies each WAL segment to storage as it fills. To restore, you take the base backup and replay the archived WAL up to a chosen time. The archive must be complete, so a missing segment breaks recovery.

PITR

Point in time recovery lets you set a target such as `recovery_target_time = 2026-11-10 09:14:00`. After an accidental delete at 09:15 you restore to 09:14 into a separate server, copy out the lost rows and keep production running. Practise the steps so they are not new during an incident.

Tools

pgBackRest and similar tools handle full, differential and incremental backups, compression, encryption, parallel restore and retention. Managed services provide snapshots and PITR, but you still choose retention, test restores and protect backups from deletion by the same account that runs the database.

Drills and monitoring

Restore a backup into a scratch environment on a schedule and run integrity checks and a sample query. Record how long it took and compare it with the recovery time objective. Alert on archive failures, the age of the last successful backup and the size of the WAL archive.

Diagram: Backups, point in time recovery and restore testing

Part	What it does	What goes wrong
Primary	Writes WAL.	Archive fails.
WAL archive	Continuous copy.	Gap in segments.
Base backups	Full copies.	Old or missing.
Restore drill	Proves it works.	Never tested.

Flow: Primary to WAL archive / Primary to Base backups / Base backups to Restore drill

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A restore that stopped at a missing segment

Incident: *During a recovery the database refused to continue because one WAL segment was missing.*

```
$ pgbackrest --stanza=main check
ERROR: [082]: WAL segment 0000000100000A3B000000F2 was not archived before the 60000ms timeout
$ pgbackrest --stanza=main info | grep -A2 "wal archive min/max"
        wal archive min/max (15): 0000000100000A3B000000A1/0000000100000A3B000000F1
(this is a simulated illustration, not a real incident)
```

The archive command had been failing for two days because a credential expired and nobody was alerted. The team renewed the credential, took a fresh full backup to cover the gap and added an alert on archive errors and on the age of the newest archived segment.

Common mistakes

- Treating replication as a backup
- Backups stored in the same account and region
- Never restoring them
- No alert on archive failures
- Retention that is too short for late discovered mistakes

Check yourself

Why is a replica not a backup?

Answer: It copies deletions immediately

What does PITR need?

Answer: A base backup and a continuous WAL archive

What should a restore drill measure?

Answer: Whether it works and how long it takes

What should be monitored?

Answer: Archive errors and the age of the last backup

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Connection pooling and capacity

12 minutes. Size connections and memory, use PgBouncer pooling modes correctly, and recognise saturation before applications see errors.

A database does not slow down gradually when connections grow: it hits a wall. Pooling and capacity limits keep you on the right side of it.

What you will learn

- Why connections are expensive
- Pool modes: session, transaction and statement
- Sizing the pool and max_connections
- Memory: shared buffers, work_mem and per connection costs
- Saturation signals

Cost of connections

In PostgreSQL each connection is a process that uses memory and competes for CPU. Thousands of idle connections waste memory, and many active ones cause context switching. Applications that open a connection per request or per pod multiply the problem, especially during scale out.

Pool modes

PgBouncer sits between applications and the database. In session mode a client keeps a server connection until it disconnects. In transaction mode it gets one only for the duration of a transaction, which allows thousands of clients to share a few dozen server connections. Transaction mode breaks features that need session state, such as session level advisory locks, `SET` and unnamed prepared statements without care.

Sizing

The number of useful active connections is roughly a small multiple of the CPU cores, often between two and four times. More just queues. Set the pool size to that number, leave room for administrators and replication, and set `max_connections` above the total of all pools plus a margin.

Memory

`shared_buffers` is often set to about a quarter of RAM. `work_mem` is allocated per sort or hash operation per connection, so a large value with many connections can exhaust memory. Raise it for specific queries or roles, not globally, and watch for the operating system killing the database.

Saturation signals

Watch active versus idle connections, connections waiting in the pool, query queue time, lock waits and CPU. The first sign is rising wait time in the pool while the database CPU is moderate. Alerts should fire on waiting clients, not only on errors.

Diagram: Connection pooling and capacity

Part	What it does	What goes wrong
Application pods	Many clients.	Pool per pod.
PgBouncer	Shares connections.	Wrong mode.
PostgreSQL	Limited backends.	Memory exhaustion.
Pool metrics	Waiting clients.	Not watched.

Flow: Application pods to PgBouncer / PgBouncer to PostgreSQL / PostgreSQL to Pool metrics

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Too many connections after a scale out

Incident: *Doubling the application pods made the database refuse new connections.*

```
$ psql -c "select state, count(*) from pg_stat_activity group by state"
 state  | count
 idle   | 870
 active | 24
$ psql -c "show max_connections"
 max_connections
 1000
(this is a simulated illustration, not a real incident)
```

Each pod kept ten idle connections and the new pods pushed the total to the limit while only 24 were doing work. PgBouncer in transaction mode with a pool of 40 connections replaced the per pod pools, and the application settings were changed to open fewer connections.

Common mistakes

- Raising max_connections instead of pooling
- Transaction pooling with session state features
- Large global work_mem
- No alert on waiting clients
- A pool per pod with no total limit

Check yourself

Why are many connections costly?

Answer: Each is a process with memory and CPU overhead

What does transaction mode allow?

Answer: Many clients sharing few server connections

How big should the active pool be?

Answer: A small multiple of CPU cores

What is a good early alert?

Answer: Clients waiting for a pool connection

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Zero downtime migrations and upgrades

13 minutes. Change schemas without locking tables using expand and contract, run safe index builds and plan major version upgrades with pg_upgrade or logical replication.

A schema change that takes a lock on a busy table is an outage. Safe migrations split a change into small compatible steps, and major upgrades need a rehearsed plan.

What you will learn

- Locks taken by common DDL
- Expand and contract
- Safe patterns: indexes, columns and constraints
- Major version upgrades
- Rollback planning

Locks

Many `ALTER TABLE` operations take an exclusive lock, and a long running query ahead of them in the queue blocks everything behind. Set `lock_timeout` so a migration fails fast instead of piling up waiters, and run it when the table is quiet. Know whether each statement rewrites the table.

Expand and contract

First expand: add the new column or table, nullable, without removing anything. Deploy code that writes both and reads the old. Backfill in small batches. Then switch reads to the new structure. Finally contract: remove the old column once nothing uses it. Each step is safe to roll back.

Safe patterns

Create indexes with `CREATE INDEX CONCURRENTLY`, which avoids blocking writes but takes longer and can leave an invalid index if it fails. Add a column with a default in recent versions without a rewrite. Add a foreign key or check constraint as `NOT VALID` and validate it afterwards, which needs a weaker lock.

Major upgrades

`pg_upgrade` upgrades in place quickly, especially with link mode, but needs a window and a tested rollback. Logical replication builds a new version cluster in parallel and lets you switch with seconds of downtime. After the upgrade, run `ANALYZE` to rebuild statistics and watch query plans.

Rollback

Before a risky step write the rollback. For schema changes it is often the reverse step. For upgrades keep the old cluster intact until the new one has been in use long enough. Rehearse on a recent copy of production data and measure the timing.

Diagram: Zero downtime migrations and upgrades

Part	What it does	What goes wrong
Expand	Add compatible structure.	Blocking lock.
Migrate code and data	Dual write, backfill.	Big batch.
Contract	Remove old structure.	Still used.
Major upgrade	Parallel cluster.	No rollback.

Flow: Expand to Migrate code and data / Migrate code and data to Contract / Contract to Major upgrade

MARGA by EknathaLabs marga.eknathalabs.com

Incident: An index build that blocked writes for ten minutes

Incident: *A plain index creation on a busy table held a lock and orders could not be saved.*

```
$ psql -c "select pid, wait_event_type, query from pg_stat_activity where wait_event_type = 'Lock' \
     limit 2"
 pid  | wait_event_type | query
 4411 | Lock            | insert into orders (...)
 4412 | Lock            | insert into orders (...)
(this is a simulated illustration, not a real incident)
```

The team created the index without CONCURRENTLY, which blocks writes for the whole build. The next time they used CREATE INDEX CONCURRENTLY with a lock timeout, checked for an invalid index afterwards and scheduled it outside the peak.

Common mistakes

- Migrations with no lock timeout
- Dropping a column before code stops using it
- Creating indexes without CONCURRENTLY on busy tables
- Upgrading with no rehearsal
- Forgetting ANALYZE after an upgrade

Check yourself

What is expand and contract?

Answer: Add compatible structure, migrate, then remove the old

Why set lock_timeout?

Answer: So a migration fails instead of blocking everything

What does CONCURRENTLY do?

Answer: Builds an index without blocking writes

What helps a major upgrade with little downtime?

Answer: Logical replication to a new cluster

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Redis and MongoDB operations

12 minutes. Operate Redis and MongoDB: persistence, memory and eviction for Redis, replica sets, sharding and indexes for MongoDB, and the failure modes of each.

Not every system uses PostgreSQL. Redis and MongoDB appear in most estates, and each has its own durability and capacity rules that operators must know.

What you will learn

- Redis persistence: RDB and AOF
- Redis memory, eviction and big keys
- Redis replication, Sentinel and Cluster
- MongoDB replica sets and write concern
- MongoDB indexes, sharding and backups

Redis persistence

Redis keeps data in memory. RDB snapshots write the dataset at intervals and are compact but can lose recent writes. The append only file logs each write and can fsync every second, which limits loss to about a second. Many caches use no persistence, and the team must know which data can be lost.

Memory and eviction

Set `maxmemory` below the machine memory with room for fork and copy on write during snapshots. The eviction policy decides what happens when it is full: `allkeys-lru` suits caches, while `noeviction` makes writes fail, which is right for data you must not lose. Find big keys and unbounded lists with `redis-cli --bigkeys` or memory analysis.

Redis availability

A primary with replicas can be managed by Sentinel, which monitors and promotes a replica, or by Redis Cluster, which shards keys over many primaries by hash slot. Replication is asynchronous, so a failover can lose recent writes. Clients must handle redirections and reconnects.

MongoDB replica sets

A replica set has a primary and secondaries, and elects a new primary when it fails with a majority vote, so use three or more voting members. Write concern `majority` waits until most members have the write, which protects it from rollback. Read preference controls whether reads may go to secondaries.

Indexes, sharding and backups

Queries without an index scan the whole collection. Check slow queries with the profiler and `explain`, and keep indexes few and useful. Sharding splits a collection across shards by a shard key. A poor key causes hot spots, so choose one with high cardinality and even distribution. Back up with snapshots or `mongodump` consistent with the oplog, and test restores.

Diagram: Redis and MongoDB operations

Part	What it does	What goes wrong
Applications	Read and write.	No reconnect logic.
Redis	Memory store.	Eviction or loss.
MongoDB replica set	Primary and secondaries.	Rollback.
Backups and monitoring	Snapshots and alerts.	Untested.

Flow: Applications to Redis / Applications to MongoDB replica set / MongoDB replica set to Backups and monitoring

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A cache that evicted the sessions

Incident: *Users were logged out when Redis reached its memory limit and removed session keys.*

```
$ redis-cli info memory | grep -E "used_memory_human|maxmemory_human|maxmemory_policy"
used_memory_human:7.98G
maxmemory_human:8.00G
maxmemory_policy:allkeys-lru
(this is a simulated illustration, not a real incident)
```

Sessions and cache entries shared one instance with an eviction policy meant for caches, so sessions were removed under pressure. The team separated sessions into their own instance with noeviction and alerts at eighty percent memory, and left the cache with allkeys-lru.

Common mistakes

- Mixing caches and primary data in one Redis
- No maxmemory with snapshots enabled
- Two member MongoDB replica sets
- A shard key with low cardinality
- No tested restore

Check yourself

What does allkeys-lru do?

Answer: Evicts least recently used keys when memory is full

Why use three MongoDB members?

Answer: To elect a primary with a majority

What does write concern majority give?

Answer: Protection from rollback after failover

What is a good shard key property?

Answer: High cardinality and even distribution

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Database incidents and runbooks

12 minutes. Respond to common database incidents with a method: classify, stabilise, diagnose with the right views, and write runbooks that others can follow.

Database incidents follow patterns: too many connections, a runaway query, locks, full disk, replication broken. A short method and good runbooks make them routine.

What you will learn

- A first five minute checklist
- Diagnosing load: queries, locks and waits
- Disk, WAL and replication emergencies
- Safe actions and what not to do
- Writing and testing runbooks

First five minutes

Confirm the impact, check whether the database is up and accepting connections, check disk and CPU, then look at connections and the longest running queries. Announce the incident and pick a commander. Avoid changing several things at once, because you will not know what worked.

Queries, locks and waits

In PostgreSQL `pg_stat_activity` shows running queries and their wait events, and `pg_locks` shows who blocks whom. A single long transaction can hold locks and prevent cleanup, which causes bloat. In MongoDB `currentOp` and the profiler play the same role. Find the blocker, not only the victims.

Disk and replication

A full disk can stop a database. Find what is large: tables, temporary files, WAL or logs. Do not delete WAL files by hand. Free space by removing safe files, dropping unused slots or extending the volume. If a replica breaks, rebuild it from a base backup rather than guessing at repairs.

Safe actions

Cancel a query with `pg_cancel_backend` before terminating it with `pg_terminate_backend`. Kill the blocker only when you understand the effect. Do not restart a database during recovery unless you must, and never run a destructive command on the only copy of the data without a backup.

Runbooks

A good runbook starts with the symptom, lists the checks in order with exact commands, states what is safe, has a decision for each result and names who to call. Test it by letting someone who did not write it follow it in a drill, then fix what was unclear.

Diagram: Database incidents and runbooks

Part	What it does	What goes wrong
Symptom	Alert or report.	Misread.
Diagnose	Queries, locks, disk.	Changing many things.
Safe action	Cancel, free space.	Destructive step.
Runbook update	Captures lessons.	Never updated.

Flow: Symptom to Diagnose / Diagnose to Safe action / Safe action to Runbook update

Incident: One idle transaction that blocked everything

Incident: *A forgotten transaction held a lock for six hours and queries stacked up behind it.*

MARGA by EknathaLabs marga.eknathalabs.com

```
$ psql -c "select pid, now()-xact_start as age, state, left(query,40) from pg_stat_activity where  \
    state = 'idle in transaction' order by age desc limit 2"
 pid  | age      | state               | left
 3902 | 06:11:02 | idle in transaction | update accounts set ...
(this is a simulated illustration, not a real incident)
```

A script opened a transaction and then waited for input. The commander cancelled it after checking it was safe, queries drained within a minute and an idle in transaction timeout was set, with an alert on the age of the oldest transaction.

Common mistakes

- Changing many things at once
- Deleting WAL files by hand
- Killing sessions without understanding them
- Runbooks that nobody has tried
- No timeout for idle in transaction sessions

Check yourself

What should you do first in an incident?

Answer: Confirm impact and check basic health

How do you find who blocks whom?

Answer: pg_locks with pg_stat_activity

What should you never do?

Answer: Delete WAL files by hand

How do you test a runbook?

Answer: Let someone else follow it in a drill

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Database operations

PostgreSQL checks, backup rules and safe schema changes.

PostgreSQL checks

Job	Query or command
Replication lag	`select now() - pg_last_xact_replay_timestamp()`
Slots	`select * from pg_replication_slots`
Active queries	`select pid, state, query from pg_stat_activity`
Blockers	`select * from pg_blocking_pids(PID)`
Cancel query	`select pg_cancel_backend(PID)`

Backups

Item	Rule
RPO	Sets WAL archive and backup frequency
RTO	Sets restore automation
Drill	Restore on a schedule and time it

Safe DDL

Change	Pattern
Index	`CREATE INDEX CONCURRENTLY`
Constraint	Add `NOT VALID` then `VALIDATE`
Lock wait	`SET lock_timeout = '5s'`

MARGA by EknathaLabs
marga.eknathalabs.com